# Harshana Lakshara Fernando

Panadura, Sri Lanka | +94 783 310 520 | harshanalf.work@gmail.com | Portfolio | LinkedIn | GitHub | Medium

## Summary

aBackend-focused Computer Science Engineering undergraduate with a rigorous foundation in **System Design**, **Distributed Systems**, and **Cloud-Native Engineering**. Proven experience architecting high-performance, fault-tolerant applications using **Microservices**, **Serverless Architectures**, and **Event-Driven Design** with **Java (Spring Boot)**, **Go**, **AWS Lambda**, and **Kafka**. Validated through the delivery of high-throughput systems handling massive load, seeking to solve complex **concurrency** and **low-latency** challenges within high-velocity engineering teams.

## Education

**B.Sc. (Hons) in Engineering – Computer Science and Engineering** *2022 – 2026*
*University of Moratuwa, Sri Lanka*

**G.C.E. Advanced Level (Physical Science Stream)** *2019*
*St. John's College, Panadura, Sri Lanka*
**Results:** 3 A's (Combined Mathematics, Physics, Chemistry) **District Rank:** 17

## Experience

**Software Engineer — Trading Systems** *Sep 2026 – Present*
*CAL | Colombo, Sri Lanka*

- Sole engineer on CAL's in-house electronic trading platform: a direct gateway to the **Colombo Stock Exchange (CSE)** plus the trading screens and ops tools around it, so the firm owns its platform instead of a vendor-built broker system.
- Took over **CSE certification** part-way and carried it to the final stage: every application-level case of the **188** exchange test cases re-run on CSE's test system with a full exchange-message log; only failover and trade-bust cases that CSE must trigger remain.
- Built an **event-sourced, CQRS** engine: a **Chronicle Queue** write-ahead log is the source of truth on the order path, with **PostgreSQL** as an async read model, so database writes never block execution.
- Implemented **FIX 5.0 SP2 / FIXT 1.1** order routing and drop copy (**QuickFIX/J**), a custom **Java NIO** decoder for the **MITCH** UDP market-data feed, and client-side token-bucket rate limiting.
- Built and own the Certification Runner (matches each test case to live exchange messages and explains every field difference from CSE's expected result), an order-lifecycle view (New, Fill, Cancel, Amend, Reject, Bust), the ops console, Pro dealing ticket and CAL Trader, plus durable message logs and an audit-ready test register.
- Using AI-assisted engineering (Claude), one person covers what is normally a team's work; design and code passed two senior reviews. Next: live CSE demonstration and sign-off, then Phase 2 (zero-allocation parsing with **Agrona**, active-passive standby).

**R&D Software Engineer — KōreroCare Platform** *Mar 2026 – Aug 2026*
*Empathic Computing Laboratory, University of Auckland | Remote / Part-time*

- Co-led backend design for the **KōreroCare** mobile-wearable sensing platform, building **Go/Gin**-based services for time-series ingestion and cross-device synchronization.
- Designed **PostgreSQL** schemas for participants, studies, devices, consent, reminders, and wearable sensor streams.
- Integrated **Keycloak OAuth2/OIDC** with **Traefik** gateway routing and Docker Compose deployment for secure internal service access.
- Developed Firebase notification and reminder flows for participant engagement and study prompts.
- Modeled an **HL7/FHIR**-aligned data schema mapping participants, devices, and wearable measurements.

**Co-founder & Lead Engineer (Freelance)** *Jul 2025 – Aug 2026*
*ScaleX — SaaS Development Lab* *Remote / Sri Lanka*

- **Stream-Native Tipping Platform:** Architecting a creator-focused SaaS to solve Sri Lankan payout barriers by integrating PayHere's aggregated gateway. Developed an OBS-native alert system using webhook-driven `notify_url` callbacks to trigger real-time on-stream overlays.
- **High-Performance Scheduling SaaS:** Engineering a mobile-first scheduling tool using Next.js 14 and Supabase Realtime for live heatmap synchronization. Implemented a custom best-slot recommendation algorithm to rank optimal meeting times based on participant overlap and duration constraints.
- **Scalable Infrastructure & DevOps:** Designed a $0-to-launch deployment strategy leveraging Vercel serverless functions and Supabase PostgreSQL with connection pooling. Established robust security protocols including Zod-based input validation and rate-limiting to ensure system resilience.
- **Strategic Compliance & Growth:** Navigating CBSL regulatory frameworks to operate as a software platform while delegating fund-handling to licensed gateways to minimize operational risk. Implementing a "freemium" viral loop strategy with branded footers and Stripe-integrated Pro tiers for monetization.
- **AgriNova:** Built a full-stack inventory and commission management platform (**Next.js**, **Prisma**, **Supabase**/**PostgreSQL**) replacing spreadsheet workflows: 5 branches, 329+ SKUs, 194K+ inventory units tracked and 860+ invoices processed (Rs. 64.4M+) with automated commission calculations. Cut repeat dashboard data-fetch latency from ∼384ms to under 1ms with React Query caching and pagination.

**Software Engineer Intern — Information Systems Associates (Pvt) Ltd** *Dec 2024 – Jun 2025*
*Aviation Technology | Colombo, Sri Lanka*

- **AeroOrder — Distributed PNR Generation Service**
  - Owned design and delivery of a distributed **Spring Boot**/**Kafka**-backed PNR generation service, **live in production**, achieving **25+ PNRs/sec** at **<100ms latency**.
  - Resolved production **Kafka consumer lag** and offset reset issues using shutdown hooks, retry handling, validation, and a Producer-Consumer refactor.
  - Led the legacy-to-microservice PNR rollout using database-driven **feature toggles** and a **strangler-pattern** migration approach.
  - Built load-testing and monitoring support using **JMeter**, **Prometheus**, and **Grafana** to validate async order/PNR flows, queue time, and service latency.
  - Maintained **>90% unit test coverage** while coordinating delivery with QA and distributed engineering teams.
- **AeroMart — Legacy Reservation System Integration**
  - Integrated **gRPC** clients and **Kafka** consumers into the **Java 8** AeroMart monolith for backward-compatible communication with AeroOrder services.
- **Engineering Practices**
  - Authored comprehensive design docs, sequence diagrams, and integration plans; facilitated knowledge transfer sessions with distributed engineering teams.
  - Added **Jenkins** environment support and streamlined service logging by replacing heavyweight logging dependencies with a lightweight structured JSON logging approach.

## Open Source Contributions — See all

- **kgateway #14851** (CNCF) *Merged, Oct 2026*
  Added the **kgateway_build_info** Prometheus metric (version, revision, Go version, OS/arch) so operators can see which build each pod runs; shipped in **kgateway v2.5.0-beta.2**.
- **kubernetes-mcp-server #1482** *Merged, Oct 2026*
  Fixed flaky HTTP server startup in the **Go** test suite with a 5 s readiness poll and cleanup before failing.
- **kubernetes-mcp-server #1549** *Merged, Oct 2026*
  Test that every tool, prompt and resource across all 12 toolsets declares valid **Kubernetes RBAC** metadata; approved on the first review.
- **Kubeflow Spark Operator #3249** *In review (LGTM)*
  20 **Ginkgo** specs covering every executor pod-phase transition: **100%** of `sparkpod_metrics.go`, plus a log message fix.
- **KEDA #8317** (CNCF) *In review*
  New **onlyReadyPods** option for the Kubernetes workload scaler (count only Running and Ready pods); design agreed with a maintainer, docs in keda-docs #1919.
- **kubernetes-mcp-server #1568** *Root-caused, Oct 2026*
  Showed a reported "flaky" test was a real merge-order bug; the author confirmed and applied the suggested

fix.

## All Projects

**AgriNova Inventory & Commission Platform (2025 | Freelance, ScaleX)** **Live demo**

Designed and delivered a production-grade, multi-branch inventory and sales management system for a real-world agricultural business, replacing manual workflows with a centralized, data-driven platform supporting inventory operations, financial tracking, and performance analytics.

- Architected a location-aware inventory model and a transaction-safe stock movement system using atomic database operations to prevent race conditions and data corruption.
- Engineered core business logic for stock updates (Issue, Return, Purchase) and a dynamic commission calculation engine based on delayed payment rules.
- Optimized performance by replacing application-layer aggregation with **PostgreSQL** aggregations (GROUP BY, COUNT, SUM), reducing location summary execution time from **∼384 ms to ∼0.02 ms**.
- Designed a multi-layer caching strategy with **Next.js** and **React Query**, reducing repeated dashboard response time from **∼703 ms to ∼0.05 ms**.
- Added composite indexes and validated execution plans using `EXPLAIN ANALYZE`, ensuring query efficiency under cache misses.
- Built a data-driven sales dashboard prioritizing clarity over visual complexity, with fully dynamic, API-driven rendering derived strictly from backend capabilities.
- Ensured system reliability by building end-to-end test coverage with **Playwright** and validating UAT workflows simulating real business operations.

*Keywords:* Next.js (App Router), PostgreSQL (Supabase), Prisma ORM, React Query, Tailwind CSS, Playwright, Vercel

**Employee Management System (EMS) (2025 | Individual)** **Architecture | Github**

Designed and built a comprehensive, cloud-native microservices application for employee and department management, featuring a full-stack architecture with a React/TypeScript frontend and Spring Boot backend.

- Architected a distributed system with independent microservices for employees, departments, and configuration, using **Spring Cloud** for service discovery (**Eureka**), centralized configuration, and an API gateway.
- Implemented robust fault tolerance using **Resilience4j**, incorporating **Circuit Breaker**, **Retry**, and **Rate Limiter** patterns to ensure system stability during partial failures.
- Enabled dynamic configuration management using **Spring Cloud Config Server** and **Spring Cloud Bus** with **RabbitMQ** for real-time property updates across services.
- Established a robust observability stack, integrating **Zipkin/Sleuth** for distributed tracing, the **ELK Stack** for centralized logging, and **Prometheus/Grafana** for real-time metrics and monitoring.
- Containerized all services using **Docker** and orchestrated local deployments with **Docker Compose**, while targeting **Kubernetes (OKE)** for production.
- Designed and managed a CI/CD pipeline using **GitHub Actions**, with **ArgoCD** for GitOps-based deployments to the Kubernetes cluster.

*Keywords:* Java 17, Spring Boot, Spring Cloud, React, TypeScript, Vite, Nginx, RabbitMQ, Resilience4j, MySQL, Docker, Kubernetes, GitHub Actions, ArgoCD, Prometheus, Grafana, ELK Stack, Nexus, GHCR

**GoTogether – Intelligent Travel Companion for Sri Lanka (2024 | Group) Architecture | Live | Github**

Developed and deployed a full-stack travel assistant platform tailored for Sri Lankan tourism, offering AI-based itinerary generation, real-time transport data, and social interaction features through a cloud-native microservices architecture.

- Led development of the web frontend using **Next.js** with secure authentication, session handling via cookies, SSR/CSR hybrid rendering, and route protection for authenticated access.
- Designed and implemented the **User Management module** within the **Social Media Service (Spring Boot)** to manage profiles, preferences, and social connections, integrating securely with **Keycloak** for identity management.
- Built a dedicated **Auth Service using Keycloak**, managing OAuth2/OIDC flows, token issuance, and automated realm/client boot-time import scripts.
- Solved startup dependency issues between services by writing a custom wait script to delay the social media service until Keycloak initialization completed, improving reliability.
- Configured **HTTPS for production with NGINX + Certbot + DuckDNS**, ensuring compatibility with Vercel's secure API policies.

- Developed the **API Service (Go)** to provide AI-based itinerary generation (via Gemini API) and map services using **gRPC** for high-performance inter-service communication.
- Optimized container builds using multi-stage Docker builds (Go service) and manual JAR injection (Spring Boot service) to reduce image size and speed up builds.
- Created a robust **deploy.sh script** to support local builds, selective service orchestration, and GitHub Actions-based CI/CD.
- Migrated backend infrastructure from **AWS EC2 to Oracle Cloud Free Tier**, improving stability, memory headroom, and deployment speed.

*Keywords:* Next.js, React Native, Go, Spring Boot, Kong, Keycloak, PostgreSQL, Cloudflare, Vercel, NGINX, Certbot, DuckDNS, Gemini API

**Food Delivery Application (2024 | Group)** **Github**

Built a scalable, event-driven food delivery platform inspired by systems like Uber Eats, using a modern microservices architecture. As part of the core team, I focused on designing and developing the Restaurant Service.

- Developed the Restaurant Module with secure sign-up/login using **Spring Security** and **JWT** (with extended claims).
- Built CRUD APIs for menu and item management; integrated **Redis caching** to improve performance and reduce database load, with appropriate cache invalidation strategies.
- Enabled order sync via **Kafka**: when an order is placed, an event is published to the 'order_created' topic, which the restaurant service consumes to update order details.
- Implemented real-time order state updates through Kafka's publish/subscribe model to keep restaurant and rider services in sync.
- Integrated **WebSockets** for real-time restaurant notifications (e.g., new orders, rider assignments).
- Built a secure "Forget Password" flow using time-sensitive tokens for authentication recovery.
- Implemented robust exception handling and API protection through middleware guards and JWT verification.
- Dockerized all components (frontend, backend, Kafka, Redis, PostgreSQL) and managed them via **Docker Compose**.
- Worked in Agile sprints using **Jira** for task tracking and **Bitbucket** for version control.

*Keywords:* Java, Spring Boot, gRPC, Kafka, Redis, PostgreSQL, WebSocket, Docker, React.js, Ant Design

**Serverless Auction Platform (2025 | Individual)** **Architecture | Github**

A capstone project built using AWS, following a microservices architecture with an event-driven, serverless (FaaS) design.

- Architected an event-driven microservices system (Auctions, Auth, Notifications) using **AWS Lambda**, **API Gateway**, and **EventBridge** for scheduled tasks.
- Implemented asynchronous service communication using **SQS** and **SNS** to decouple components and handle email notifications via **SES**.
- Designed a high-performance data model in **DynamoDB** utilizing **Global Secondary Indexes (GSI)** for efficient scheduled querying and **On-Demand Capacity** for auto-scaling.
- Secured API endpoints with a custom **Lambda Authorizer** that validates **JSON Web Tokens (JWT)**, ensuring robust role-based access control.
- Enhanced code reliability and maintainability by integrating **Middy** middleware for input validation and error handling.
- Utilized **LocalStack** and **Docker** to create a comprehensive local development environment that emulates AWS cloud infrastructure.

*Keywords:* Node.js, Serverless Framework, AWS Lambda, DynamoDB, SQS, SNS, SES, JWT, Middy, LocalStack, Docker

**Automated AI Shorts Video Creator (2025 | Individual)**

An open-source project to automate the creation of short-form videos using a powerful combination of AI tools and workflow automation.

- Orchestrated a video creation pipeline using **n8n** as the central workflow engine.
- Integrated various AI services for script generation, text-to-speech, and content preparation, including **Together AI**, **OpenRouter**, and **ElevenLabs**.
- Containerized the entire stack using **Docker Compose** for one-click local deployment.
- Implemented notifications via **Telegram** to monitor the automation workflow.

*Keywords:* n8n, Docker, AI, Workflow Automation, Video Generation

**AI Blog Post Generator (2025 | Individual)**

A multi-agent system that uses specialized AI agents to generate professional, well-structured, and well-researched blog posts.

- Built with the **Agno Framework** for its beginner-friendly approach to multi-agent workflows.
- Orchestrated research, content extraction, and writing using specialized agents for each task.
- Designed for provider flexibility, with easy switching between **OpenAI, Gemini, Claude, and Groq** models.
- Implemented a user-friendly interface with **Streamlit**, including real-time streaming output.

*Keywords:* Streamlit, Agno Framework, Pydantic

**Maison Vella – E-Commerce Admin Panel (2024 | Individual) Architecture | Live | Github**

Comprehensive admin interface for the Maison Vella e-commerce ecosystem, facilitating centralized management of products, orders, and content assets.

- Engineered a responsive **Next.js 14** application using the **App Router**, **ShadCN UI**, and **Tailwind CSS**, ensuring a consistent and accessible design system.
- Implemented a robust backend architecture utilizing **Server Actions** for form handling and **tRPC** with **TanStack Query** for type-safe client-server communication.
- Orchestrated content management by integrating **Sanity** as a **Headless CMS**, consuming data via **GraphQL** for flexible and efficient retrieval.
- Managed transactional data using **Prisma ORM** with **Supabase (PostgreSQL)** in a **Serverless** environment.
- Architected the system using **CQRS**, the **Repository Pattern**, and a **Layered Architecture** within a feature-sliced structure to ensure scalability and maintainability.
- Secured administrative access and role-based operations using **Clerk Auth**.
- Optimized asset delivery and storage by implementing **Cloudflare R2** (S3-compatible) and a global **CDN** for high-performance media handling.

*Keywords:* Next.js 14 (App Router), Server Actions, TypeScript, tRPC, GraphQL, Prisma, Supabase, Sanity (Headless CMS), Serverless, Cloudflare R2 (CDN), CQRS

**Go gRPC GraphQL – Multi-Tenant Microservices (2025 | Individual)**

Architected a **polyglot, multi-tenant microservices** platform with a focus on scalability, fault isolation, and unified data access.

- Developed backend services in **Go** using **gRPC** and **Protobuf** for efficient inter-service communication.
- Implemented a **GraphQL API Gateway** to provide a single, unified endpoint for client applications.
- Designed multi-tenant data storage with **PostgreSQL**, **Elasticsearch**, and optional **Cassandra** for high-volume workloads.
- Containerized and orchestrated services using **Docker Compose** for local and staging environments.

*Keywords:* Go, gRPC, GraphQL, Protobuf, PostgreSQL, Elasticsearch, Cassandra, Docker Compose

**Fintech & Crypto Trading Automation Lab (Ongoing | Group)**

*Industry-Mentored Initiative (Ongoing)*

Selected to participate in an exclusive, industry-mentored initiative focused on cutting-edge development in fintech, virtual assets, and derivative markets.

- Selected based on strong development skills to work on intelligent automation and production-level projects with real-world deadlines.
- Developing high-performance trading automation tools and fintech solutions with opportunities for mutual ownership.
- Gaining deep exposure to technical analysis (TA), algorithmic trading, and asset management strategies across crypto and forex markets.
- Receiving direct industrial mentorship aimed at long-term growth in personal finance, wealth management, and professional software engineering practices.

*Keywords:* Fintech, Crypto Trading, Forex, Algorithmic Trading, Technical Analysis, Intelligent Automation, Industrial Mentorship, Personal Finance & Wealth Management

**Improving Blockchain Scalability: Throughput Enhancement in ZK-Rollups**

*Research Project — BSc (Hons) in Computer Science & Engineering, University of Moratuwa*

- Designed and implemented a modular ZK-rollup benchmarking framework to evaluate Ethereum Layer-2 performance trade-offs across sequencing, batching, proving, data availability, and Layer-1 settlement.

- Built an end-to-end benchmark pipeline consisting of a Workload Generator, Sequencer, Executor, Prover, Rust-based Submitter, Solidity L1 contracts, and Python data-analysis tools.
- Implemented and refactored the Rust Submitter service to receive finalized L2 batches, select the configured Data Availability mode, submit commitments to the L1 bridge contract, validate transaction receipts, and export settlement metrics.
- Added configurable Data Availability paths including calldata, EIP-4844-style blob mode, and off-chain / Validium-style benchmark mode, enabling controlled comparison of cost, latency, and DA-footprint trade-offs.
- Introduced a mock-testable `BridgeClient` abstraction for L1 interactions, improving Submitter testability and increasing verifiable Submitter test coverage from approximately 41% to 88%.
- Fixed a critical settlement reliability issue by validating L1 transaction receipt status, preventing mined but reverted transactions from being incorrectly recorded as successful benchmark results.
- Contributed Solidity bridge, Data Availability, and verifier-related contracts to provide a local L1 settlement boundary for end-to-end benchmark execution.
- Developed Docker-based benchmark orchestration scripts for service startup, repeated experiment execution, workload triggering, result management, and metrics extraction.
- Built a synthetic Poisson-style workload generator supporting controlled traffic profiles, including an 8 TPS base workload with 80 TPS burst injection for stress testing batching and scheduling behaviour.
- Developed Python data tools to aggregate JSON/JSONL/CSV metrics and compute throughput, P50/P95/P99 latency, finalization time, gas/cost summaries, and DA footprint.
- Generated reproducible benchmark outputs including latency CDFs, cost heatmaps, and Pareto-style trade-off comparisons for the final research evaluation.
- Demonstrated that EIP-4844-style blob data availability reduced data publication cost by approximately 70% compared with calldata in the benchmark environment.

*Key Results:* Delivered a working end-to-end ZK-rollup benchmark workflow from workload generation to L1 submission and post-experiment analysis; improved Submitter test coverage from approximately 41% to 88%; evaluated 8 TPS base and 80 TPS burst workloads; observed approximately 70% DA publication cost reduction using blob-style DA compared with calldata.
*Technologies:* Rust, Solidity, Hardhat, TypeScript, Python, Docker, gRPC, SQLite/PostgreSQL, Ethereum Local Test Networks, RISC0 / Mock Proving Path, JSON/JSONL/CSV Metrics, Matplotlib

**Blockchain Supply Chain Management DApp (2025 | Individual)** **Architecture | Live | Github**

Built a full-stack decentralized application (DApp) enabling end-to-end shipment tracking and secure escrow payments on the Ethereum blockchain (Polygon Amoy Testnet).

- Developed a production-grade Solidity smart contract (`Tracking.sol`) using `structs`, `mappings`, and a strict 3-state `enum` lifecycle (PENDING → IN_TRANSIT → DELIVERED), complete with guarded state transitions, timestamped event logs, and full on-chain auditability.
- Implemented a trustless escrow mechanism using `payable` functions—validating `msg.value`, locking funds on creation, and releasing payments only upon verified delivery via `completeShipment`.
- Built a responsive Next.js/React frontend powered by a global `TrackingContext`, providing real-time state management, shipment inspection, and lifecycle actions (create, start, complete).
- Integrated `Ethers.js` and `Web3Modal` for secure wallet connection, dynamic provider/signer selection, MetaMask interaction, and seamless network handling (Localhost Polygon Amoy).
- Added data-formatting utilities for user-friendly display, including UNIX→date conversion and Ether/Wei transformations using `ethers.utils.formatEther`.
- Implemented polished UI/UX behaviors—loading indicators, masked modals (`maskClosable=false`), and utility helpers (`shortenAddress`, `copyToClipboard`) for high-quality user interaction.
- Designed a manual CI/CD pipeline using GitHub Actions to automate contract deployment (with pre-flight balance checks) and frontend deployment to Vercel, fully parameterized via GitHub Secrets and environment isolation.
- Utilized Hardhat for development, including local blockchain simulation (`npx hardhat node`), compilation, contract verification, and controlled deployment workflows.

*Keywords:* Solidity, Hardhat, Ethers.js, Web3Modal, Next.js, React Context API, Tailwind CSS, Polygon Amoy, Smart Contracts, Escrow Payments, CI/CD

**Governance DAO – Token-Weighted Voting DApp (2025 | Individual)**

A decentralized autonomous organization (DAO) voting application on Ethereum.

- Developed **Solidity** smart contracts for proposal creation and token-weighted voting.
- Implemented an ERC-20 governance token for voting power.
- Built a frontend using **React** and **Web3.js** to interact with the blockchain.
- Deployed contracts to the **Sepolia** testnet using **Hardhat**.

- Ensured security through comprehensive unit tests and access control modifiers.

*Keywords:* Solidity, Ethereum, React, Web3.js, DAO, Smart Contracts, Hardhat

**Content-Aware Meme Generator (2025 | Individual)**

An AI-powered tool that generates memes based on image context.

- Utilized **OpenAI GPT-4 Vision** to analyze images and generate humorous captions.
- Integrated **Pillow (PIL)** for image processing and text overlay.
- Created a simple web interface using **Streamlit** for user uploads.
- Implemented error handling and rate limiting for API requests.

*Keywords:* Python, OpenAI Vision API, Pillow, Streamlit, AI, Image Processing

**PaperMind – AI Research Assistant (2025 | Individual)**

Developed a lightweight, **RAG-powered** platform for intelligent querying of academic papers.

- Enabled PDF uploads for local semantic search and integrated with **arXiv** for external paper retrieval.
- Designed a **LangChain** pipeline with **ChromaDB** vector storage for fast, context-aware lookups.
- Generated embeddings using **Sentence Transformers** for improved retrieval accuracy.
- Delivered LLM-generated responses using **Gemini API** and HuggingFace models.

*Keywords:* Streamlit, LangChain, ChromaDB, Sentence Transformers, Gemini API, HuggingFace

**RPAL Interpreter – Functional Language Compiler (2024 | Group)**

Engineered a complete compiler and interpreter for the RPAL (Right-reference Pedagogic Algorithmic Language), built from scratch to parse and evaluate programs according to official language grammar and semantics.

- Developed a Lexical Analyzer using custom Finite State Automata (FSA) and regex-based token identification.
- Built an Abstract Syntax Tree (AST) and Standard Tree (ST) generator.
- Implemented a CSE Machine to evaluate programs using 13 semantic rules of RPAL.
- Built a robust testing framework with **Pytest** covering over 168 test cases across AST, ST, and full output validation.
- Automated build and test pipelines using **Makefiles** and **GitHub Actions**.
- Supports interactive CLI switches (-ast, -st, -ct, -t, -ft) for granular output inspection.

*Keywords:* Python, Pytest, GitHub Actions, Makefile, CLI

**Virtual Me – 3D Animated AI Chatbot (2024 | Individual)**

Created an interactive 3D virtual assistant with lifelike animations and AI-driven conversations.

- Integrated **Hugging Face** (Mixtral model) for natural language dialogue.
- Implemented **Eleven Labs** TTS for human-like voice output.
- Synchronized lip movements and facial expressions using **Rhubarb**.
- Built immersive 3D interface with **React** and **Three.js**.
- Developed backend using **Node.js** and **Express.js** for API orchestration and audio processing.

*Keywords:* React (Three.js), Node.js, Express.js, Hugging Face, Eleven Labs, Rhubarb, FFmpeg

## Training & Certifications

CKA Training — KodeKloud, 2025 • AWS SAA-C03 Training — Udemy, 2025 • Apache Kafka Series — Udemy, 2025 • 100 Days of DevOps — KodeKloud, 2025 • Advanced OAuth Security — Udemy, 2024

## Skills

| | |
|---|---|
| **Core** | **Java** \| **Spring Boot** \| **Microservices** \| Distributed Systems \| System Design \| **REST APIs** |
| **Backend** | Spring Cloud \| Spring Security \| Resilience4j \| Node.js \| Express.js \| Gin |
| **Messaging & APIs** | **Kafka** \| RabbitMQ \| **gRPC** \| **GraphQL** \| WebSockets \| Protobuf \| JWT \| OAuth 2.0 |
| **Data** | **PostgreSQL** \| MySQL \| Oracle DB \| Redis \| Couchbase |
| **Cloud & DevOps** | AWS \| **Docker** \| Kubernetes \| CI/CD (GitHub Actions, Jenkins) |
| **Testing** | JMeter \| Gatling \| Pytest \| Unit Testing |
| **Tools** | Git \| Gradle \| Postman \| Jira \| Confluence |

*References available upon request.*